

## Advanced Collections: Map, Set, WeakMap & WeakSet

Chapter 11-এ আমরা **Prototype, Prototype Chain, Constructor, Class & Inheritance** শিখেছি। এবার JavaScript-এর এমন কিছু data structure শিখব যেগুলো real-life application-এ অনেক কাজে লাগে:

Map → Set → WeakMap → WeakSet

বিশেষ করে **Map** এবং **Set** খুব ভালোভাবে শেখা জরুরি! 🔥

## PART 01 — MAP: DEEP DIVE & PRACTICAL APPLICATIONS (12.1 – 12.26)

### 12.1 Collection কী?

Collection মানে হলো:

একসাথে অনেকগুলো data সুন্দরভাবে সংরক্ষণ করার structure।

আমরা আগে Array এবং Object শিখেছি।

যেমন:

12.1 JAVASCRIPT

RUN

```
const students = ["Ripon", "Rahim", "Karim"];
```

এখানে Array ব্যবহার করেছি।

আবার:

12.1 JAVASCRIPT

RUN

```
const student = {  
  name: "Ripon",  
  department: "CSE"  
};
```

এখানে Object ব্যবহার করেছি।

কিন্তু কিছু ক্ষেত্রে Object/Array-এর চেয়ে **Map** বা **Set** বেশি সুবিধাজনক।

## 12.2 Map কী?

Map হলো এমন একটি collection যেখানে:

```
key → value
```

pair হিসেবে data রাখা হয়।

সহজভাবে:

```
Map

key      value
-----
name    →   Ripon
age     →    25
dept    →    CSE
```

Object-এর মতো মনে হচ্ছে, কিন্তু Map অনেক বেশি flexible!

## 12.3 Map তৈরি করা

12.3 JAVASCRIPT

RUN

```
const user = new Map();

console.log(user);
```

```
Map(0) {}
```

## 12.4 Map-এ Data যোগ করা — set()

Map-এ data যোগ করতে:

12.4 JAVASCRIPT

RUN

```
set()
```

ব্যবহার করা হয়।

12.4 JAVASCRIPT

RUN

```
const user = new Map();

user.set("name", "Ripon");
user.set("age", 25);
user.set("department", "CSE");

console.log(user);
```

```
Map(3) {
  "name" => "Ripon",
  "age" => 25,
  "department" => "CSE"
}
```

### 12.5 Map থেকে Data নেওয়া — `get()`

12.5 JAVASCRIPT

RUN

```
const user = new Map();

user.set("name", "Ripon");
user.set("age", 25);

console.log(user.get("name"));
console.log(user.get("age"));
```

#### CONSOLE OUTPUT

Ripon  
25

#### ★ মনে রাখো:

`set()` → data রাখে  
`get()` → data বের করে

### 12.6 Map-এ Key না থাকলে

12.6 JAVASCRIPT

RUN

```
const user = new Map();

user.set("name", "Ripon");

console.log(user.get("age"));
```

#### CONSOLE OUTPUT

undefined

কারণ:

TEXT  
age

key নেই।

### 12.7 `has()` — Key আছে কিনা

12.7 JAVASCRIPT

RUN

```
const user = new Map();

user.set("name", "Ripon");

console.log(user.has("name"));
console.log(user.has("age"));
```

#### CONSOLE OUTPUT

true  
false

#### ⚙️ SYNTAX:

12.7 JAVASCRIPT

RUN

```
map.has(key);
```

### 12.8 delete() — Data Remove করা

12.8 JAVASCRIPT

RUN

```
const user = new Map();

user.set("name", "Ripon");
user.set("age", 25);

user.delete("age");

console.log(user);
```

```
Map(1) {
  "name" => "Ripon"
}
```

### 12.9 clear() — সব Data Remove

12.9 JAVASCRIPT

RUN

```
const user = new Map();

user.set("name", "Ripon");
user.set("age", 25);

user.clear();

console.log(user);
```

```
Map(0) {}
```

### 12.10 size — কতগুলো Item আছে

12.10 JAVASCRIPT

RUN

```
const user = new Map();

user.set("name", "Ripon");
user.set("age", 25);
user.set("department", "CSE");

console.log(user.size);
```

CONSOLE OUTPUT

3

খেয়াল করো:

12.10 JAVASCRIPT

RUN

```
map.size
```

এখানে `()` নেই।

### 12.11 Map-এর Basic Methods

| Method                | কাজ                       |
|-----------------------|---------------------------|
| <code>set()</code>    | Data যোগ করে              |
| <code>get()</code>    | Data বের করে              |
| <code>has()</code>    | Key আছে কিনা দেখে         |
| <code>delete()</code> | নির্দিষ্ট item remove করে |
| <code>clear()</code>  | সব item remove করে        |
| <code>size</code>     | মোট item সংখ্যা           |

### 12.12 Map-এর Key শুধু String হতে হবে না

এটাই Map-এর বড় সুবিধাগুলোর একটি।

Object-এ সাধারণত property key string বা symbol-based হয়।

কিন্তু Map-এ key হিসেবে ব্যবহার করা যায়:

```
TEXT
String
Number
Boolean
Object
Array
Function
Symbol
```

ইত্যাদি।

### 12.13 Number Key

12.13 JAVASCRIPT

RUN

```
const students = new Map();

students.set(101, "Ripon");
students.set(102, "Rahim");

console.log(students.get(101));
```

CONSOLE OUTPUT

Ripon

### 12.14 Boolean Key

12.14 JAVASCRIPT

RUN

```
const data = new Map();

data.set(true, "Logged In");
data.set(false, "Logged Out");

console.log(data.get(true));
```

CONSOLE OUTPUT

Logged In

### 12.15 Object as Map Key

🔥 খুব গুরুত্বপূর্ণ!

12.15 JAVASCRIPT

RUN

```
const user1 = {
  name: "Ripon"
};

const user2 = {
  name: "Rahim"
};

const scores = new Map();

scores.set(user1, 95);
scores.set(user2, 88);

console.log(scores.get(user1));
console.log(scores.get(user2));
```

CONSOLE OUTPUT

95  
88

এখানে object-গুলো নিজেরাই key!

### 12.16 Array as Map Key

12.16 JAVASCRIPT

RUN

```
const map = new Map();

const numbers = [1, 2, 3];

map.set(numbers, "Numbers Array");

console.log(map.get(numbers));
```

CONSOLE OUTPUT

Numbers Array

### 12.17 Function as Map Key

12.17 JAVASCRIPT

RUN

```
function greet() {
  console.log("Hello");
}

const map = new Map();

map.set(greet, "Greeting Function");

console.log(map.get(greet));
```

CONSOLE OUTPUT

Greeting Function

### 12.18 Map Constructor দিয়ে সরাসরি Data

Map তৈরি করার সময় initial data দেওয়া যায়।

12.18 JAVASCRIPT

RUN

```
const user = new Map([
  ["name", "Ripon"],
  ["age", 25],
  ["department", "CSE"]
]);

console.log(user.get("name"));
```

CONSOLE OUTPUT

Ripon

### 12.19 Map-এর উপর Loop

Map-এর সবচেয়ে common way:

12.19 JAVASCRIPT

RUN

for ... of

💡 EXAMPLE

12.19 JAVASCRIPT

RUN

```
const user = new Map([
  ["name", "Ripon"],
  ["age", 25],
  ["department", "CSE"]
]);

for (const [key, value] of user) {
  console.log(key, value);
}
```

CONSOLE OUTPUT

name Ripon  
age 25  
department CSE

### 12.20 keys()

শুধু keys দেখতে:

12.20 JAVASCRIPT

RUN

```
const user = new Map([
  ["name", "Ripon"],
  ["age", 25],
  ["department", "CSE"]
]);

for (const key of user.keys()) {
  console.log(key);
}
```

CONSOLE OUTPUT

name  
age  
department

### 12.21 values()

সমস্ত values:

12.21 JAVASCRIPT

RUN

```
for (const value of user.values()) {  
  console.log(value);  
}
```

#### CONSOLE OUTPUT

Ripon  
25  
CSE

### 12.22 entries()

Key + value pair:

12.22 JAVASCRIPT

RUN

```
for (const entry of user.entries()) {  
  console.log(entry);  
}
```

#### CONSOLE OUTPUT

["name", "Ripon"]  
["age", 25]  
["department", "CSE"]

### 12.23 Map Destructuring

এভাবে সবচেয়ে সুন্দর:

12.23 JAVASCRIPT

RUN

```
for (const [key, value] of user) {  
  console.log(`${key}: ${value}`);  
}
```

#### CONSOLE OUTPUT

name: Ripon  
age: 25  
department: CSE

### 12.24 Map-এর Real-Life Example — Student Marks

ধরো Student ID দিয়ে marks store করব।

12.24 JAVASCRIPT

RUN

```
const marks = new Map();  
  
marks.set("CSE101", 85);  
marks.set("CSE102", 90);  
marks.set("CSE103", 78);  
  
console.log(marks.get("CSE102"));
```

#### CONSOLE OUTPUT

90

এখানে:

Student ID → Marks



## 12.25 Map-এর Real-Life Example — Product Price

12.25 JAVASCRIPT

RUN

```
const products = new Map();

products.set("Laptop", 75000);
products.set("Phone", 35000);
products.set("Keyboard", 2500);

console.log(products.get("Laptop"));
```

CONSOLE OUTPUT

75000

## 12.26 Map ব্যবহার করে Frequency Counter

এটি খুব গুরুত্বপূর্ণ এবং interview-এও কাজে লাগে।

ধরো:

12.26 JAVASCRIPT

RUN

```
const fruits = [
  "apple",
  "banana",
  "apple",
  "orange",
  "banana",
  "apple"
];
```

আমরা count করতে চাই।

12.26 JAVASCRIPT

RUN

```
const count = new Map();

for (const fruit of fruits) {
  count.set(
    fruit,
    (count.get(fruit) || 0) + 1
  );
}

console.log(count);
```

```
Map(3) {
  "apple" => 3,
  "banana" => 2,
  "orange" => 1
}
```

🔥 এই pattern অনেক useful:

CONSOLE OUTPUT

```
map.set(
  key,
  (map.get(key) || 0) + 1
);
```

**12.27 Set কী?**এখন আসি `Set`-এ।**Set হলো এমন একটি collection যেখানে duplicate value রাখা হয় না।****EXAMPLE:**

12.27 JAVASCRIPT

RUN

```
const numbers = new Set();

numbers.add(10);
numbers.add(20);
numbers.add(10);

console.log(numbers);
```

```
Set(2) {
  10,
  20
}
```

দুইবার `10` দেওয়া হলেও একবারই আছে।**12.28 Set তৈরি করা**

12.28 JAVASCRIPT

RUN

```
const numbers = new Set();

console.log(numbers);
```

```
Set(0) {}
```

**12.29 add()**

Set-এ data যোগ করতে:

12.29 JAVASCRIPT

RUN

```
numbers.add(10);
numbers.add(20);
numbers.add(30);
```

**12.30 Duplicate Data**

12.30 JAVASCRIPT

RUN

```
const numbers = new Set();

numbers.add(10);
numbers.add(10);
numbers.add(10);

console.log(numbers.size);
```

CONSOLE OUTPUT

1

**12.31** `has()`

12.31 JAVASCRIPT

RUN

```
const numbers = new Set([10, 20, 30]);  
  
console.log(numbers.has(20));  
console.log(numbers.has(50));
```

**CONSOLE OUTPUT**

```
true  
false
```

**12.32** `delete()`

12.32 JAVASCRIPT

RUN

```
const numbers = new Set([10, 20, 30]);  
  
numbers.delete(20);  
  
console.log(numbers);
```

```
Set(2) {  
  10,  
  30  
}
```

**12.33** `clear()`

12.33 JAVASCRIPT

RUN

```
const numbers = new Set([10, 20, 30]);  
  
numbers.clear();  
  
console.log(numbers);
```

```
Set(0) {}
```

**12.34** `size`

12.34 JAVASCRIPT

RUN

```
const numbers = new Set([10, 20, 30]);  
  
console.log(numbers.size);
```

**CONSOLE OUTPUT**

```
3
```

### 12.35 Set Constructor দিয়ে Array

12.35 JAVASCRIPT

RUN

```
const numbers = new Set([10, 20, 30, 40]);  
  
console.log(numbers);
```

```
Set(4) {  
  10,  
  20,  
  30,  
  40  
}
```

### 12.36 Array থেকে Duplicate Remove

🔥 Set-এর সবচেয়ে common practical use!

ধরো:

12.36 JAVASCRIPT

RUN

```
const numbers = [  
  10,  
  20,  
  10,  
  30,  
  20,  
  40  
];
```

Duplicate remove:

12.36 JAVASCRIPT

RUN

```
const uniqueNumbers = [...new Set(numbers)];  
  
console.log(uniqueNumbers);
```

CONSOLE OUTPUT

```
[10, 20, 30, 40]
```

### 12.37 Real-Life Example — Unique Skills

12.37 JAVASCRIPT

RUN

```
const skills = [
  "HTML",
  "CSS",
  "JavaScript",
  "React",
  "JavaScript",
  "Node.js",
  "React"
];

const uniqueSkills = [ ...new Set(skills)];

console.log(uniqueSkills);
```

CONSOLE OUTPUT

```
[
  "HTML",
  "CSS",
  "JavaScript",
  "React",
  "Node.js"
]
```

এটা real-life portfolio বা form data cleaning-এ কাজে লাগতে পারে।

### 12.38 Set-এর উপর Loop

12.38 JAVASCRIPT

RUN

```
const skills = new Set([
  "HTML",
  "CSS",
  "JavaScript"
]);

for (const skill of skills) {
  console.log(skill);
}
```

CONSOLE OUTPUT

```
HTML
CSS
JavaScript
```

### 12.39 Set-এর `keys()` , `values()` , `entries()`

Set-এর ক্ষেত্রে:

12.39 JAVASCRIPT

RUN

```
set.keys()
set.values()
set.entries()
```

আছে।

#### 💡 EXAMPLE

12.39 JAVASCRIPT

RUN

```
const numbers = new Set([10, 20, 30]);

console.log([ ... numbers.keys()]);
console.log([ ... numbers.values()]);
console.log([ ... numbers.entries()]);
```

#### CONSOLE OUTPUT

```
[10, 20, 30]
```

```
[10, 20, 30]
```

```
[
  [10, 10],
  [20, 20],
  [30, 30]
]
```

Set-এ key এবং value একই।

### 12.40 Set কি Object-এর মতো?

না।

#### Object

12.40 JAVASCRIPT

RUN

```
const user = {
  name: "Ripon",
  age: 25
};
```

Key-value pair:

```
key → value
```

#### Set

12.40 JAVASCRIPT

RUN

```
const skills = new Set([
  "HTML",
  "CSS",
  "JS"
]);
```

সুধু unique values:

TEXT

```
value
value
value
```

#### 12.41 Map বনাম Object

খুব গুরুত্বপূর্ণ comparison!

| বিষয়                | Object                | Map          |
|----------------------|-----------------------|--------------|
| Key-value            | ✓                     | ✓            |
| Key type             | String/Symbol         | যেকোনো value |
| size                 | সরাসরি নেই            | ✓            |
| set()                | ✗                     | ✓            |
| get()                | ✗                     | ✓            |
| has()                | আলাদা methods         | ✓            |
| Iteration            | তুলনামূলক less direct | সহজ          |
| Object key           | special handling      | সরাসরি       |
| Structured data      | খুব ভালো              | ভালো         |
| Dictionary-like data | ভালো                  | খুব ভালো     |

#### 12.42 কখন Object ব্যবহার করব?

যখন data-এর structure fixed বা JSON/API-like হয়:

12.42 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon",  
  email: "ripon@example.com",  
  department: "CSE"  
};
```

এখানে Object খুব natural!

#### 12.43 কখন Map ব্যবহার করব?

যখন:

- Dynamic keys আছে
- Key হিসেবে object/array/function লাগবে
- Frequent add/remove দরকার
- Size জানতে হবে
- Key-value collection নিয়ে বেশি কাজ করতে হবে

তখন Map ভালো choice হতে পারে।

#### 12.44 Set বনাম Array

| বিষয়         | Array                | Set      |
|---------------|----------------------|----------|
| Duplicate     | ✓                    | ✗        |
| Index         | ✓                    | ✗        |
| length        | ✓                    | ✗        |
| size          | ✗                    | ✓        |
| push()        | ✓                    | ✗        |
| add()         | ✗                    | ✓        |
| Unique values | নিজে handle করতে হয় | Built-in |

**12.45 WeakMap কী?**

এখন advanced part।

**WeakMap** দেখতে Map-এর মতো, কিন্তু এর key-এর ক্ষেত্রে গুরুত্বপূর্ণ restriction আছে:

**WeakMap-এর key অবশ্যই object বা non-registered Symbol হতে হবে।**

সবচেয়ে সহজভাবে object key দিয়ে example দেখি।

12.45 JAVASCRIPT

RUN

```
const weakMap = new WeakMap();

const user = {
  name: "Ripon"
};

weakMap.set(user, "Private Data");

console.log(weakMap.get(user));
```

CONSOLE OUTPUT

Private Data

**12.46 WeakMap-এ String Key দেওয়া যাবে?**

12.46 JAVASCRIPT

RUN

```
const weakMap = new WeakMap();

weakMap.set("name", "Ripon");
```

এটি কাজ করবে না।

কারণ:

WeakMap key → Object / non-registered Symbol

**12.47 WeakMap-এর Basic Methods**

WeakMap-এ:

```
set()
get()
has()
delete()
```

আছে।

কিন্তু Map-এর মতো:

```
size
keys()
values()
entries()
clear()
```

নেই।



#### 12.48 WeakMap কেন "Weak"?

এখানে memory management-এর ধারণা আসে।

ধরো:

12.48 JAVASCRIPT

RUN

```
let user = {  
  name: "Ripon"  
};  
  
const weakMap = new WeakMap();  
  
weakMap.set(user, "Private Data");
```

এখন `user` object-এর আর কোনো strong reference না থাকলে garbage collector ভবিষ্যতে object-টিকে memory থেকে remove করতে পারে।

WeakMap সেই object-কে জোর করে memory-তে ধরে রাখে না।

এজন্য:

**WeakMap = weak reference behavior**

#### 12.49 Garbage Collection-এর Basic Idea

JavaScript automatically unused objects-এর memory reclaim করতে পারে।

ধরো:

12.49 JAVASCRIPT

RUN

```
let user = {  
  name: "Ripon"  
};
```

তারপর:

12.49 JAVASCRIPT

RUN

```
user = null;
```

যদি object-টির আর কোনো reachable reference না থাকে, garbage collector ভবিষ্যতে সেটিকে remove করতে পারে।

এটি JavaScript-এর automatic memory management-এর অংশ।

## 12.50 WeakMap Real-Life Use

WeakMap ব্যবহার করা যায় object-এর সাথে extra/private metadata রাখার ক্ষেত্রে।

12.50 JAVASCRIPT

RUN

```
const privateData = new WeakMap();

class User {
  constructor(name, password) {
    this.name = name;

    privateData.set(this, {
      password: password
    });
  }

  checkPassword(password) {
    return privateData.get(this).password === password;
  }
}

const user = new User(
  "Ripon",
  "12345"
);

console.log(user.checkPassword("12345"));
```

CONSOLE OUTPUT

true

এখানে password সরাসরি:

12.50 JAVASCRIPT

RUN

```
user.password
```

হিসেবে রাখা হয়নি।

## 12.51 WeakSet কী?

**WeakSet** হলো Set-এর weak-reference version।

এখানে সাধারণত object রাখা হয়।

12.51 JAVASCRIPT

RUN

```
const weakSet = new WeakSet();

const user1 = {
  name: "Ripon"
};

const user2 = {
  name: "Rahim"
};

weakSet.add(user1);
weakSet.add(user2);

console.log(weakSet.has(user1));
```

CONSOLE OUTPUT

true

### 12.52 WeakSet-এ Primitive রাখা যাবে?

না।

12.52 JAVASCRIPT

RUN

```
const weakSet = new WeakSet();

weakSet.add(10);
```

এটি কাজ করবে না।

WeakSet-এ object বা non-registered Symbol ধরনের value ব্যবহার করা যায়।

### 12.53 WeakSet-এর Methods

মূল methods:

TEXT

```
add()
has()
delete()
```

WeakSet -এ:

```
size
keys()
values()
entries()
clear()
```

নেই।

### 12.54 WeakMap বনাম Map

| বিষয়                       | Map          | WeakMap                        |
|-----------------------------|--------------|--------------------------------|
| Key                         | যেকোনো value | Object / non-registered Symbol |
| get()                       | ✓            | ✓                              |
| set()                       | ✓            | ✓                              |
| has()                       | ✓            | ✓                              |
| delete()                    | ✓            | ✓                              |
| size                        | ✓            | ✗                              |
| Iteration                   | ✓            | ✗                              |
| Garbage collection friendly | সাধারণ       | বেশি                           |
| Object metadata             | ভালো         | খুব useful                     |

### 12.55 WeakSet বনাম Set

| বিষয়       | Set          | WeakSet                        |
|-------------|--------------|--------------------------------|
| Values      | যেকোনো value | Object / non-registered Symbol |
| Duplicate   | ✗            | ✗                              |
| add()       | ✓            | ✓                              |
| has()       | ✓            | ✓                              |
| delete()    | ✓            | ✓                              |
| size        | ✓            | ✗                              |
| Iteration   | ✓            | ✗                              |
| GC-friendly | সাধারণ       | বেশি                           |

**12.56 Map + Set Practical Project**

এবার একটা ছোট বাস্তব example করি।

ধরো আমরা একটা online course platform বানাচ্ছি।

একজন user কোন কোন course enroll করেছে তা রাখতে চাই।

12.56 JAVASCRIPT

RUN

```
const enrolledCourses = new Map();

const user1 = {
  id: 101,
  name: "Ripon"
};

enrolledCourses.set(user1, new Set([
  "JavaScript",
  "React",
  "Node.js"
]));

console.log(
  enrolledCourses.get(user1)
);
```

```
Set(3) {
  "JavaScript",
  "React",
  "Node.js"
}
```

এখানে:

```
Map
  ↓
User → Courses

Set
  ↓
Unique Courses
```

**12.57 User-এর Course Check**

12.57 JAVASCRIPT

RUN

```
const courses = enrolledCourses.get(user1);

console.log(courses.has("React"));
console.log(courses.has("Python"));
```

**CONSOLE OUTPUT**

```
true
false
```

### 12.58 Course Add করা

12.58 JAVASCRIPT

RUN

```
courses.add("Python");  
  
console.log(courses);
```

```
Set(4) {  
  "JavaScript",  
  "React",  
  "Node.js",  
  "Python"  
}
```

### 12.59 Duplicate Course Add

12.59 JAVASCRIPT

RUN

```
courses.add("React");  
  
console.log(courses.size);
```

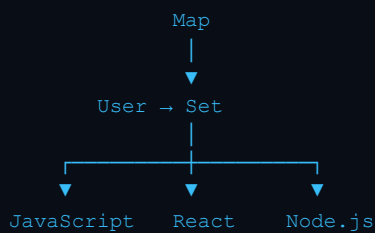
CONSOLE OUTPUT

4

কারণ React আগেই ছিল।

### 12.60 Map + Set Mental Model

এটা মনে রাখার মতো:



এ ধরনের nested data structure real application-এ ব্যবহার করা যায়।

**MUST KNOW** 🔥 12.61 Must Know

Chapter 12 থেকে অবশ্যই জানতে হবে:

**Map**

JAVASCRIPT

RUN

```
const map = new Map();

map.set("name", "Ripon");
map.get("name");
map.has("name");
map.delete("name");
map.clear();

map.size;
```

**Set**

JAVASCRIPT

RUN

```
const set = new Set();

set.add("JavaScript");
set.has("JavaScript");
set.delete("JavaScript");
set.clear();

set.size;
```

**Duplicate Remove**

JAVASCRIPT

RUN

```
const unique = [...new Set(array)];
```

**WeakMap**

JAVASCRIPT

RUN

```
const weakMap = new WeakMap();

weakMap.set(object, value);
weakMap.get(object);
weakMap.has(object);
weakMap.delete(object);
```

**WeakSet**

JAVASCRIPT

RUN

```
const weakSet = new WeakSet();

weakSet.add(object);
weakSet.has(object);
weakSet.delete(object);
```

## CHEAT SHEET

## 🗨 Quick Cheat Sheet

Object

- Key-value data
- সাধারণ structured data

Map

- Key-value data
- Any type of key
- Dynamic collection

Set

- Unique values
- Duplicate automatically removed

WeakMap

- Object/non-registered Symbol keys
- Weak reference behavior
- No iteration/size

WeakSet

- Object/non-registered Symbol values
- Weak reference behavior
- No iteration/size

## DECISION GUIDE

## 🎯 কখন কোনটা ব্যবহার করব?

আমার data যদি...

Key → Value

↓

Object

Dynamic Key → Value

↓

Map

শুধু Unique Values

↓

Set

Object-এর সাথে extra metadata

↓

WeakMap

Object-গুলো tracked/marked রাখতে চাই

↓

WeakSet

**Practice 1 — Map**

একটি Map বানাও:

```
101 → Ripon
102 → Rahim
103 → Karim
```

তারপর `102` দিয়ে নাম বের করো।

**Practice 2 — Set**

এই Array থেকে duplicate remove করো:

JAVASCRIPT

RUN

```
const skills = [
  "HTML",
  "CSS",
  "JS",
  "React",
  "JS",
  "CSS",
  "Node"
];
```

Expected:

TEXT

```
[
  "HTML",
  "CSS",
  "JS",
  "React",
  "Node"
]
```

**Practice 3 — Frequency Counter**

এই Array-তে প্রত্যেক number কতবার আছে বের করো:

JAVASCRIPT

RUN

```
const numbers = [
  1, 2, 3, 1, 2, 1, 4, 3
];
```

Expected:

```
1 → 3
2 → 2
3 → 2
4 → 1
```

**Practice 4 — Object as Map Key**

দুইজন user object তৈরি করো:

JAVASCRIPT

RUN

```
const user1 = {};
const user2 = {};
```

তারপর Map ব্যবহার করে তাদের score রাখো।



### Practice 5 — Map + Set

একটি Map বানাও যেখানে:

```
Student → Courses
```

যেমন:

```
Ripon → HTML, CSS, JavaScript  
Rahim → JavaScript, React
```

তারপর check করো:

TEXT

Ripon কি React course-এ আছে?

**SUMMARY** **Chapter 12 Final Summary — Collections & Memory Model**

আজ আমরা শিখলাম:

```
Map
↓
set()
get()
has()
delete()
clear()
size
keys()
values()
entries()
```

তারপর:

```
Set
↓
add()
has()
delete()
clear()
size
```

তারপর advanced:

```
WeakMap
↓
Object → Metadata
↓
Weak Reference
```

এবং:

```
WeakSet
↓
Objects track করা
↓
Weak Reference
```

সবচেয়ে গুরুত্বপূর্ণ ৪টি লাইন মনে রাখা:

JAVASCRIPT

RUN

```
const map = new Map();

const set = new Set();

const weakMap = new WeakMap();

const weakSet = new WeakSet();
```

আর practical JavaScript-এ সবচেয়ে বেশি মনে রাখার মতো দুইটি pattern:

JAVASCRIPT

RUN

```
// Duplicate remove
const unique = [...new Set(array)];
```

এবং:

JAVASCRIPT

RUN

```
// Frequency counter
map.set(key, (map.get(key) || 0) + 1);
```

**NEXT CHAPTER** **Chapter 13 Preview — Built-in Objects**

**Chapter 13-এ** আমরা এরপর **JavaScript Built-in Objects** — `Date`, `Math`, `Number`, `String`, `RegExp`, `Intl`, `BigInt` বিস্তারিতভাবে শিখতে পারি।